

GALE-SHAPLEY (preference lists for hospitals and students)

INITIALIZE M to empty matching.

WHILE (some hospital h is unmatched and hasn't proposed to every student)

$s \leftarrow$ first student on h 's list to whom h has not yet proposed.

IF (s is unmatched)

Add $h-s$ to matching M .

ELSE IF (s prefers h to current partner h')

Replace $h'-s$ with $h-s$ in matching M .

ELSE

s rejects h .

RETURN ~~stable matching~~ M .

best for hospitals
best for students

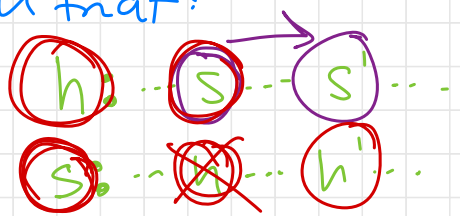
1. Once a student becomes matched, they never become unmatched. $\top$
2. Once a student becomes matched, they only trade up for better hospitals. $\top$
3. Once a student becomes matched, they only trade down for worse hospitals. $\top$
4. Once a hospital becomes matched, it never becomes unmatched. $\top$
5. A hospital only trades up for better students. $\top$
6. A hospital only trades down for worse students. $\top$
7. A hospital will never propose to the same student twice. $\top$
8. The last student a hospital proposes to is the one it ends up with. $\top$

claim The set M returned by G-S is a stable matching.

Proof Let M be the matching returned by G-S on some arbitrary input. For the sake of contradiction, assume that M is not stable — that is, there is an unstable pair (h, s) with respect to M .

This means that there are two pairs (h, s') and (h', s) such that:

- h prefers s to s'
- s prefers h to h'



By ⑧, s' was the last student h proposed to.

Notice that h prefers s to s' , so by ⑥ h must have proposed to s in the past.

By ②, students never trade down, so s never would have traded down from h to h' .

We have arrived at a contradiction, so the assumption that an unstable pair exists is false.

Claim G-S matches each hospital with its best matching student.

best?

highest ranked feasible match

$h; s_1, s_2, s_3, \dots, s_n$

a match (h, s_i) is feasible if there exists a stable matching M s.t. $(h, s_i) \in M$.

highest-ranked feasible: $\min i$.

with table: if we wanted the best M for students, what could we do?

Analysis of Algorithms

how does an algorithm scale with increasing input size?

in tables: list topics you already know about

- time complexity
- using data structures effectively
- upper bounds / lower bounds

O
 $\leq$

Ω
 $\geq$

asymptotic notation

- big Theta

Θ
 $=$

- space complexity

- trade offs

• polynomial vs non polynomial

• worst case / best / avg case